

Govind Sarthi AI — Master Application Architecture & Preview

1. Executive Summary & Core Philosophy

Govind Sarthi AI is an empathetic, user-authenticated spiritual reflection journal inspired by the timeless wisdom of the *Bhagavad Gita* and the *Mahabharata*. Built on Google Cloud Run, Firebase Authentication, Cloud Firestore, and the Gemini 3.6 Flash API with multi-model fallback ladders, it serves as a compassionate charioteer (*Sarthi*) for human conscience.

Core Directives & Values

- Absolute Truth (Satya):** Faithfully embodies Lord Krishna's wisdom—never misleading, never flattering ego, and never validating false choices or shortcuts.
- Universal Love (Prem) & Purpose:** Guides seekers away from materialistic ego (*Ahankar*) toward duty (*Dharma*), universal love, and selfless action (*Nishkama Karma*).
- Environmental Stewardship:** Instills deep reverence for Mother Earth (*Prithvi Mata*), clean air (*Pranvayu*), and natural life, motivating users to actively eliminate pollution and protect nature.
- Socratic Discovery:** Uses gentle reverse-psychology and relatable Mahabharata analogies (Arjuna's dilemma, Karna's silence, Drupada's pride, Draupadi's veneration as Shakti) to help users discover clarity from within.
- Firm Moral Authority:** Shifts instantly from soothing guidance to commanding divine reprimand when users express cruel, abusive, or illegal intent (*Adharma*).

2. Threat Modeling Summary (5 Security Zones)

Threat Zone	Identified Risks	Countermeasures & Production Mitigations
Input Surfaces	Malicious prompt injections attempting to bypass moral counsel; malformed payloads.	Strict schema enforcement; string input caps (max 8,000 chars); HTML entity sanitization prior to execution.
Planning & Reasoning	Persona hijacking or getting the agent to validate crime/violence.	Hardened system instructions establishing an immutable ethical hierarchy; conditional shift to firm moral reprimand upon detecting malevolence.
Tool Execution	Browser speech synthesis blocking, Web Audio buffer memory leaks, rate-limit failures.	Native Web Speech API integration; sequential state machine for audio ducking; resilient fallback ladder (gemini-3.6-flash → gemini-3.1-flash-lite → gemini-flash-latest → gemini-3.7-flash).
Memory & State	Cross-user reflection leaks; unauthorized database reads/writes.	Owner-bound Firestore rules (request.auth.uid == userId); Firebase Admin SDK verifying Bearer JWTs at every backend boundary.
Inter-System Comm.	Exposed API keys or hardcoded service account credentials.	Dynamic secret resolution via Google Cloud Secret Manager; zero frontend API key exposure.

3. Backend Core Implementation

server/geminiHelper.js — Resilient Gemini Client with Fallback Ladder

```
const { GoogleGenAI } = require('@google/genai');

const MODEL_FALLBACK_LADDER = [
  'gemini-3.6-flash',
  'gemini-3.1-flash-lite',
  'gemini-flash-latest',
  'gemini-3.7-flash'
];

const SARTHI_MASTER_PROMPT = `You are "Shri Govind" (Divine Sarthi / Charioteer of the Conscience), inspired b

LANGUAGE & TONE PROFILE:
- Speak in pure, authentic Shuddh Hindi (उच्च एवं शुद्ध हिंदी) as spoken in classical spiritual dialogues.
- Seamlessly integrate Sanskrit words with soft bracketed meanings for clarity (e.g., धर्म [Duty], अहंकार [Ego], ए
- Tone MUST be deeply empathetic, soft, gentle, unhurried, noble, and comforting. Never sound harsh, mechanica

CORE SPIRITUAL & PSYCHOLOGICAL DIRECTIVES:
1. Deep Active Listening: First, acknowledge the seeker's feelings with warmth. Let them release all their une
2. Perspective & Gratitude: Gently remind them that human birth is a rare reward (दुर्लभ मानव जीवन). Help them see
3. Ego vs Universal Love (अहंकार एवं प्रेम):
   - "आधी में वही वृक्ष टिकता है जो झुकना जानता है; जो कठोर होते हैं वे उखड़ जाते हैं।"
   - Show how ego destroys precious relationships. True love is unconditional, respectful of destiny (विधि का विध
4. Mahabharata Wisdom & Analogies: Naturally draw wisdom from Karna's silence during Adharma, Drupada's pride
5. Socratic Discovery & Realization: Use gentle reverse-psychology to make seekers discover their own ego and
6. Environmental & Social Duty: Inspire seekers to protect nature from pollution, help others selflessly in ti
7. Unyielding Moral Guardrail: If a user expresses cruel, abusive, or illegal intent (अधर्म), drop gentle irony
8. Conclude positive resolutions warmly with: "मेरी योजना पर विश्वास रखो (Trust my plans). Do your karma selflessly, ;

let aiClient = null;

function getAiClient() {
  if (!aiClient) {
    const apiKey = process.env.GEMINI_API_KEY;
    if (!apiKey) {
      throw new Error('GEMINI_API_KEY environment variable is missing.');
```

```

try {
  const response = await client.models.generateContent({
    model: modelName,
    contents: contents,
    config: {
      systemInstruction: SARTHI_MASTER_PROMPT,
      temperature: 0.72,
      maxOutputTokens: 1400
    }
  });

  if (response && response.text) {
    return {
      text: response.text,
      modelUsed: modelName
    };
  }
} catch (err) {
  console.warn(`[Fallback Ladder] Model ${modelName} failed: ${err.message}. Proceeding...`);
  lastError = err;
}

throw new Error(`All fallback models exhausted. Last error: ${lastError?.message || 'Unknown'}`);
}

module.exports = { generateGovindResponse };

```

server/index.js — Express API Server & Payload Hygiene

```
const express = require('express');
const cors = require('cors');
const path = require('path');
const { db, authenticateUser, admin } = require('./firebaseAdmin');
const { generateGovindResponse } = require('./geminiHelper');
const { cleanFirestorePayload, sanitizeInputString } = require('./utils/sanitizer');

const app = express();
const PORT = process.env.PORT || 8080;

app.use(cors());
app.use(express.json({ limit: '2mb' }));
app.use(express.urlencoded({ extended: true }));
app.use(express.static(path.join(__dirname, '../public')));

app.get('/health', (req, res) => {
  res.status(200).json({ status: 'healthy', timestamp: new Date().toISOString() });
});

app.get('/api/reflections', authenticateUser, async (req, res) => {
  try {
    const userId = req.user.uid;
    const snapshot = await db.collection('users').doc(userId).collection('interactions')
      .orderBy('createdAt', 'desc').limit(40).get();

    const reflections = [];
    snapshot.forEach(doc => reflections.push({ id: doc.id, ...doc.data() }));

    return res.status(200).json({ reflections });
  } catch (error) {
    console.error('Error fetching reflections:', error);
    return res.status(500).json({ error: 'Failed to retrieve reflection history.' });
  }
});

app.post('/api/reflections', authenticateUser, async (req, res) => {
  try {
    const body = (req.body && typeof req.body === 'object') ? req.body : {};
    const prompt = sanitizeInputString(body.prompt);
    const history = Array.isArray(body.history) ? body.history : [];

    if (!prompt) {
      return res.status(400).json({ error: 'Reflection prompt cannot be empty.' });
    }

    const userId = req.user.uid;
    const aiResult = await generateGovindResponse(prompt, history);

    const reflectionRecord = cleanFirestorePayload({
      prompt: prompt,
      response: aiResult.text,
      modelUsed: aiResult.modelUsed,
      userId: userId,
      createdAt: admin.firestore.FieldValue.serverTimestamp()
    });

    const docRef = await db.collection('users').doc(userId).collection('interactions').add(reflectionRecord);

    return res.status(201).json({
```

```
        id: docRef.id,
        prompt: reflectionRecord.prompt,
        response: reflectionRecord.response,
        modelUsed: reflectionRecord.modelUsed,
        createdAt: new Date().toISOString()
    });
} catch (error) {
    console.error('Reflection processing error:', error);
    return res.status(500).json({ error: 'Govind could not complete reflection. Please try again.' });
}
});

app.get('*', (req, res) => {
    res.sendFile(path.join(__dirname, '../public/index.html'));
});

app.listen(PORT, () => console.log(`Govind Sarthi Server active on port ${PORT}`));
```

4. Frontend Celestial Garden UI & Audio Architecture

public/index.html — Botanical Garden Interface Markup

```
<!DOCTYPE html>
<html lang="hi">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>श्री गोविंद – Sarthi Spiritual Reflection Garden</title>
  <link rel="stylesheet" href="styles.css">
  <script src="https://www.gstatic.com/firebasejs/10.12.0/firebase-app-compat.js"></script>
  <script src="https://www.gstatic.com/firebasejs/10.12.0/firebase-auth-compat.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/marked/marked.min.js"></script>
</head>
<body>
  <!-- 3D Botanical Petals Layer -->
  <canvas id="petals-canvas"></canvas>

  <div id="app">
    <!-- Notification Banner -->
    <div id="toast" class="toast hidden">
      <span id="toast-message"></span>
      <button id="toast-retry-btn" class="btn btn-secondary hidden">पुनः प्रयास करें (Retry)</button>
      <button id="toast-close-btn" class="btn-close">&times;</button>
    </div>

    <!-- Auth Landing Screen -->
    <section id="auth-view" class="view active">
      <div class="auth-card">
        <div class="garden-badge">🌿 श्री गोविंद सारथी</div>
        <h1>अंतरमन की शांति एवं आत्म-चिंतन</h1>
        <p>जीवन की दुविधाओं, अहंकार और अंतर्द्वंद्व को त्यागकर अपने सारथी से संवाद करें। गीता के शाश्वत ज्ञान से अपने धर्म को पहचानें।</p>

        <div class="verse-preview">
          "कर्मण्येवाधिकारस्ते मा फलेषु कदाचन। मा कर्मफलहेतुर्भूर्मा ते सङ्गोऽस्त्रवकर्मणि॥"
        </div>

        <button id="btn-google-login" class="btn btn-primary">
          <svg width="18" height="18" viewBox="0 0 24 24">
            <path fill="#4285F4" d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h5.92c-.26 1.37-1.04 2.53-2.21
            <path fill="#34A853" d="M12 23c2.97 0 5.46-.98 7.28-2.66l-3.57-2.77c-.98.66-2.23 1.06-3.71 1.06-2.
            <path fill="#FBBC05" d="M5.84 14.09c-.22-.66-.35-1.36-.35-2.09s.13-1.43.35-2.09V7.06H2.18C1.43 8.5
            <path fill="#EA4335" d="M12 5.38c1.62 0 3.06.56 4.21 1.64l3.15-3.15C17.45 2.09 14.97 1 12 1 7.7 1
          </svg>
          Google से प्रवेश करें
        </button>
      </div>
    </section>

    <!-- Private Reflection Workspace -->
    <section id="dashboard-view" class="view hidden">
      <header class="navbar">
        <div class="nav-brand">
          <span>🌿 श्री गोविंद शरणम्</span>
        </div>
        <div class="user-menu">
          <button id="btn-music-toggle" class="btn btn-secondary music-btn" title="Toggle Flute & Tanpura Ambi
            <span class="music-icon">🎵</span>
            <span id="music-btn-text">बांसुरी संगीत</span>
          </button>
        </div>
      </header>
    </section>
  </div>
</body>
</html>
```

```
<button id="btn-voice-toggle" class="btn btn-secondary" title="Voice Guidance"><img alt="voice icon" data-bbox="735 92 755 105"/> वाणी सक्रिय</button>
<span id="user-display-name">पार्थ</span>
<button id="btn-logout" class="btn btn-secondary">प्रस्थान करें</button>
</div>
</header>

<main class="dashboard-layout">
  <aside class="sidebar">
    <div class="sidebar-header">
      <h3>पूर्व चिंतन (Past Reflections)</h3>
      <button id="btn-refresh-history" class="btn btn-icon" title="Sync"><img alt="refresh icon" data-bbox="640 225 655 235"/></button>
    </div>
    <div id="history-list" class="history-list">
      <p class="empty-state">कोई पूर्व चिंतन उपलब्ध नहीं है।</p>
    </div>
  </aside>

  <section class="workspace">
    <div id="conversation-container" class="conversation-container">
      <div class="garden-welcome">
        <h2>कहो पार्थ, आज तुम्हारे हृदय में कौन सा दृढ़ चल रहा है?</h2>
        <p>बिना किसी संकोच के अपने मन का संपूर्ण भार व्यक्त करो। गोविंद तुम्हें सुन रहे हैं।</p>
      </div>

      <form id="reflection-form" class="input-form">
        <textarea
          id="reflection-input"
          placeholder="अपने मन की व्यथा, क्रोध, अपराधबोध या दुविधा यहाँ लिखें..."
          rows="3"
          required
        ></textarea>
        <div class="input-actions">
          <span id="model-indicator" class="model-badge"><img alt="gemini logo" data-bbox="245 530 260 540"/> Powered by Gemini 3.6 Flash</span>
          <button type="submit" id="btn-submit" class="btn btn-primary">गोविंद से मार्गदर्शन लें</button>
        </div>
      </form>
    </div>
  </section>
</main>
</div>

<script src="petalsAnimation.js"></script>
<script src="voiceGuide.js"></script>
<script src="app.js"></script>
</body>
</html>
```

public/voiceGuide.js — Advanced Text Sanitizer & Gentle Vocal Narration Engine

```
/**
 * Govind Sarthi Audio Manager & Text-to-Speech Engine
 * Features: Deep text sanitization (stripping devanagari punctuation, brackets, markdown),
 * gentle vocal pacing, ambient flute loop control, and audio ducking.
 */
class GovindAudioManager {
  constructor() {
    this.synth = window.speechSynthesis;
    this.voice = null;
    this.isMuted = false;
    this.isAmbientPlaying = false;

    // Divine Ambient Bansuri & Tanpura Audio Stream
    this.ambientAudio = new Audio('https://cdn.pixabay.com/download/audio/2022/05/16/audio_c89e248a39.mp3?file');
    this.ambientAudio.loop = true;
    this.ambientAudio.crossOrigin = 'anonymous';

    this.AMBIENT_DEFAULT_VOL = 0.20;
    this.AMBIENT_DUCKED_VOL = 0.04;
    this.ambientAudio.volume = this.AMBIENT_DEFAULT_VOL;

    this.shlokas = [
      {
        id: 'gita_2_47',
        name: 'Karmanye Vadhikaraste',
        sanskrit: 'कर्मण्येवाधिकारस्ते मा फलेषु कदाचन। मा कर्मफलहेतुर्भूर्मा ते सङ्गोऽस्त्रवकर्मणि॥'
      },
      {
        id: 'shanti_mantra',
        name: 'Sarve Bhavantu Sukhinah',
        sanskrit: 'ॐ सर्वे भवन्तु सुखिनः सर्वे सन्तु निरामयाः। सर्वे भद्राणि पश्यन्तु मा कश्चिद्दुःखभाग्भवेत्॥'
      }
    ];

    this.initVoice();
    this.bindVisibilityHandler();
  }

  initVoice() {
    if (!('speechSynthesis' in window)) return;
    const loadVoices = () => {
      const voices = this.synth.getVoices();
      // Select natural Hindi / Indian English voice
      this.voice = voices.find(v => v.lang.includes('hi') || v.lang.includes('hi-IN')) ||
        voices.find(v => v.lang.includes('en-IN')) ||
        voices.find(v => v.name.toLowerCase().includes('natural') || v.name.toLowerCase().includes(
          voices[0]));
    };
    loadVoices();
    if (speechSynthesis.onvoiceschanged !== undefined) {
      speechSynthesis.onvoiceschanged = loadVoices;
    }
  }

  bindVisibilityHandler() {
    document.addEventListener('visibilitychange', () => {
      if (document.hidden) {
        if (this.isAmbientPlaying) this.ambientAudio.pause();
      } else {

```

```

        if (this.isAmbientPlaying && !this.isMuted) this.ambientAudio.play().catch(() => {});
    }
});
}

toggleAmbientMusic() {
    if (this.isAmbientPlaying) {
        this.ambientAudio.pause();
        this.isAmbientPlaying = false;
        return false;
    } else {
        this.ambientAudio.volume = this.AMBIENT_DEFAULT_VOL;
        this.ambientAudio.play().then(() => { this.isAmbientPlaying = true; }).catch(() => {});
        return true;
    }
}

fadeAmbientVolume(targetVolume, durationMs = 800) {
    if (!this.isAmbientPlaying) return;
    const startVol = this.ambientAudio.volume;
    const steps = 16;
    const stepTime = durationMs / steps;
    const volChange = (targetVolume - startVol) / steps;
    let currentStep = 0;

    const interval = setInterval(() => {
        currentStep++;
        this.ambientAudio.volume = Math.max(0, Math.min(1, this.ambientAudio.volume + volChange));
        if (currentStep >= steps) {
            this.ambientAudio.volume = targetVolume;
            clearInterval(interval);
        }
    }, stepTime);
}

/**
 * Deep Text Sanitization for Speech Synthesis
 * Strips all markdown syntax, devanagari punctuation (।, ॥), brackets, symbols, and emojis
 * so the voiceover reads pure, smooth Hindi without robotic breaks or reading aloud symbols.
 */
sanitizeForSpeech(text) {
    if (!text) return '';
    return text
        .replace(/\[.*?\]/g, '') // Remove bracketed text e.g. [Duty], [Ego]
        .replace(/\(.*?\)/g, '') // Remove parenthetical notes
        .replace(/[#*`_~@$%^&+=|\\<>]/g, '') // Remove markdown & special symbols
        .replace(/\[|||:-]/g, ' ') // Replace devanagari dandas & dashes with natural breath pauses
        .replace(/[\u{1F600}-\u{1F64F}\u{1F300}-\u{1F5FF}\u{1F680}-\u{1F6FF}\u{2600}-\u{26FF}\u{2700}-\u{27BF}]/g, '') // Remove emojis
        .replace(/\s+/g, ' ') // Normalize multi-spaces
        .trim();
}

playChantAndSpeak(text, onChantStart = null) {
    if (this.isMuted) return;
    this.fadeAmbientVolume(this.AMBIENT_DUCKED_VOL, 600);

    const selectedShloka = this.shlokas[Math.floor(Math.random() * this.shlokas.length)];
    if (onChantStart) onChantStart(selectedShloka);

    setTimeout(() => {
        this.speak(text, () => {

```

```

        this.fadeAmbientVolume(this.AMBIENT_DEFAULT_VOL, 1000);
    });
}, 2800);
}

speak(rawText, onComplete = null) {
    if (this.isMuted || !this.synth) {
        if (onComplete) onComplete();
        return;
    }

    this.synth.cancel(); // Stop any ongoing speech
    const cleanText = this.sanitizeForSpeech(rawText);

    if (!cleanText) {
        if (onComplete) onComplete();
        return;
    }

    const utterance = new SpeechSynthesisUtterance(cleanText);
    if (this.voice) utterance.voice = this.voice;

    // Soothing, gentle, warm Krishna-like vocal settings
    utterance.rate = 0.82; // Slow, unhurried, peaceful tempo
    utterance.pitch = 0.98; // Natural, warm, soft pitch (non-scary)
    utterance.volume = 0.92; // Comforting volume level

    utterance.onend = () => { if (onComplete) onComplete(); };
    utterance.onerror = () => { if (onComplete) onComplete(); };

    this.synth.speak(utterance);
}

stopAll() {
    this.ambientAudio.pause();
    this.isAmbientPlaying = false;
    if (this.synth) this.synth.cancel();
}

toggleMute() {
    this.isMuted = !this.isMuted;
    if (this.isMuted) this.stopAll();
    else this.toggleAmbientMusic();
    return this.isMuted;
}
}

window.govindAudio = new GovindAudioManager();

```

5. Security Configuration (firestore.rules)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Default deny-all policy
    match /{document=**} {
      allow read, write: if false;
    }

    // Owner-bound isolation for reflections and spiritual dialogues
    match /users/{userId}/interactions/{interactionId} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }
  }
}
```

6. Test & Verification Matrix

Test ID	Scenario	User Action / Trigger	Expected Outcome
TC-01	Google OAuth Sign-In	Click "Google से प्रवेश करें".	Authenticates user; opens Vrindavan Reflection Space dashboard displaying user's name as "पार्थ".
TC-02	Gentle Speech & Symbol Stripping	Govind's response arrives with brackets and dandas.	Text sanitizer strips all dandas (,), brackets, and markdown symbols; voiceover reads smoothly in a soft, soothing tone (rate: 0.82).
TC-03	Socratic Irony & Ego Mirroring	User submits: "I shouted at my friend in class because he cracked a bad joke."	Govind responds with gentle Socratic irony, guiding the user to realize how ego damaged a sacred friendship.
TC-04	Mahabharata & Prayaschit Guidance	User admits guilt: "I broke his trust. What should I do?"	Govind provides the example of Karna and Duryodhana / Arjuna, guiding them toward true repentance (Prayaschit) and duty (Dharma).
TC-05	Smart Audio Ducking	Reflection response arrives.	Ambient music dips to 4% volume, Sanskrit shloka banner renders, and Govind's voice speaks smoothly before restoring background audio to 20%.

7. Deployment Commands for Google Cloud Run

1. Enable Required GCP Services

```
gcloud services enable \  
  run.googleapis.com \  
  secretmanager.googleapis.com \  
  firestore.googleapis.com
```

2. Store Gemini API Key in Secret Manager

```
gcloud secrets create GEMINI_API_KEY --replication-policy="automatic"  
echo -n "YOUR_GEMINI_API_KEY" | gcloud secrets versions add GEMINI_API_KEY --data-file=-
```

3. Grant Secret Accessor IAM Role to Cloud Run Service Account

```
PROJECT_NUMBER=$(gcloud projects describe YOUR_PROJECT_ID --format="value(projectNumber)")  
gcloud secrets add-iam-policy-binding GEMINI_API_KEY \  
  --member="serviceAccount:${PROJECT_NUMBER}-compute@developer.gserviceaccount.com" \  
  --role="roles/secretmanager.secretAccessor"
```

4. Deploy Application to Google Cloud Run

```
gcloud run deploy govind-sarathi-app \  
  --source . \  
  --region asia-southeast1 \  
  --allow-unauthenticated \  
  --set-secrets="GEMINI_API_KEY=GEMINI_API_KEY:latest"
```

5. Apply Mandatory Challenge Verification Label

```
gcloud run services update govind-sarathi-app \  
  --update-labels=dev-tutorial=cloud-run-ai-challenge \  
  --region asia-southeast1
```